Tutorial 2 (to be completed in week2)
Data Modeling (Part 2) & Relational Set Operators

Tutorial Objectives:

- Transform conceptual model into a logical model using J Developer
- Learn about relational set operators.

Tutorial Tasks:

a. Transform the (conceptual) Class Model into a (logical) Database Model.
b. Apply relational set operators in Oracle 12c to a given problem domain. Initial SQL script file is provided.
c. Analyse some relational operators using the given script.

Part a

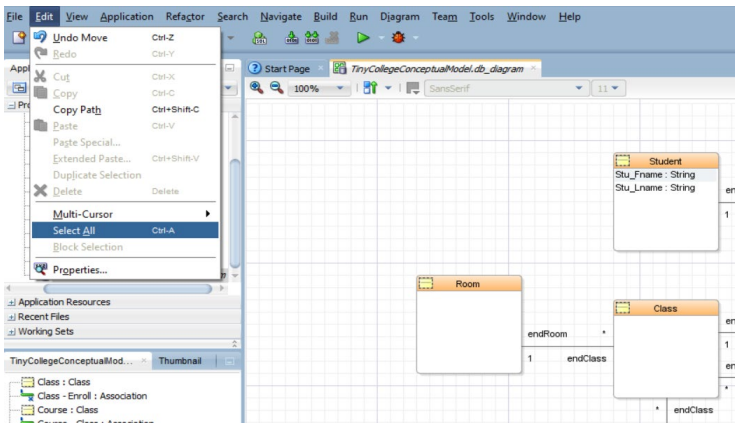
a. Transform the Class Model (Tiny College example) into a Database Model.

In Tutorial 1B, a UML class diagram (although you may have used **database-drawing** palette) was created for the Tiny College case study, using J Developer. This was a Conceptual Model with classes, associations and few attributes but did not have tables, primary keys, foreign keys, etc. In this tutorial (Part a), the same model will be used to auto-generate a Logical Model (a database model with tables, PKs, FKs, etc.). We will then modify and/or extend this generated model by altering (e.g., changing data types, sizes, etc.) and/or adding missing/further information (e.g., missing relationships, additional attributes, data types, etc.) to the diagram.

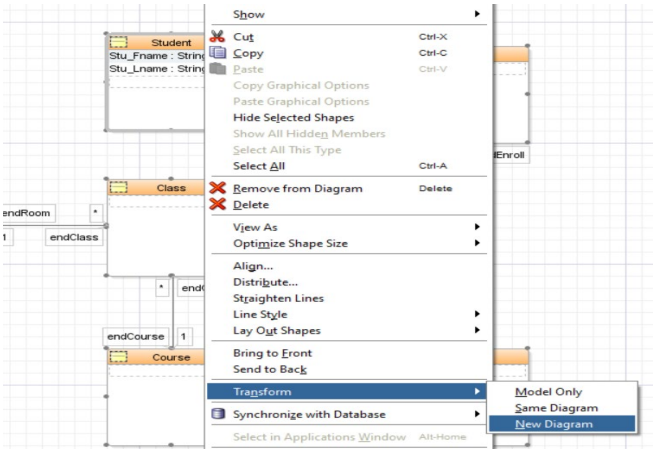
i. First, open your application file (e.g., DBModellingApplication.jws) from Tutorial 1B inside J Developer. Now open your class diagram (e.g., TinyCollegeConceptualModel) which is part of the application's project (e.g., TinyCollegeModel.jpr). Follow the steps provided under (ii.) below to transform your conceptual model into a logical one (e.g., classes will change to tables).

ii. Now Transform the Class Model into a Database Model.

   a. Now from the Main Menu select Edit --> Select All to select all classes and associations in the diagram.

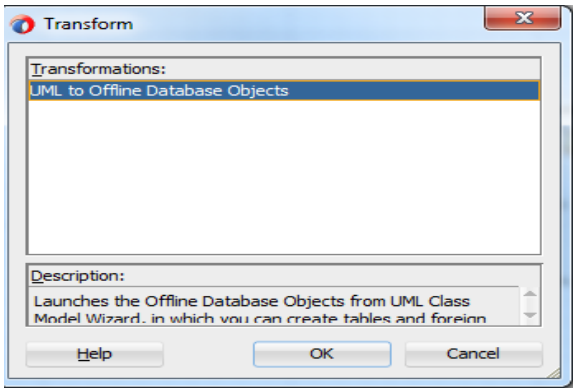


b. Right-click and from the context related menu select **Transform --> New Diagram**.

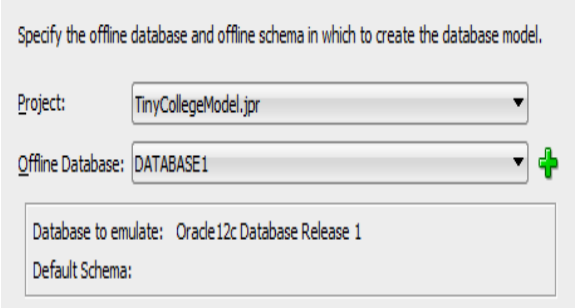


c. In the **Transform** dialog, select **UML to Offline Database Objects**.

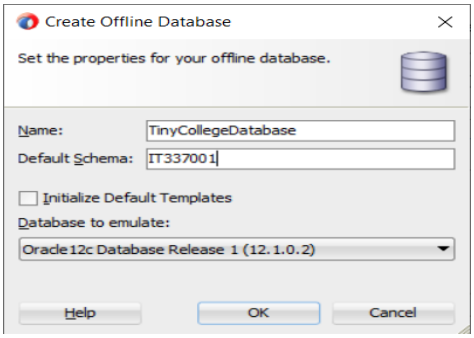
(Note: If we had opened J Developer with other than **Database Developer** role, we would then have other options to choose from.)



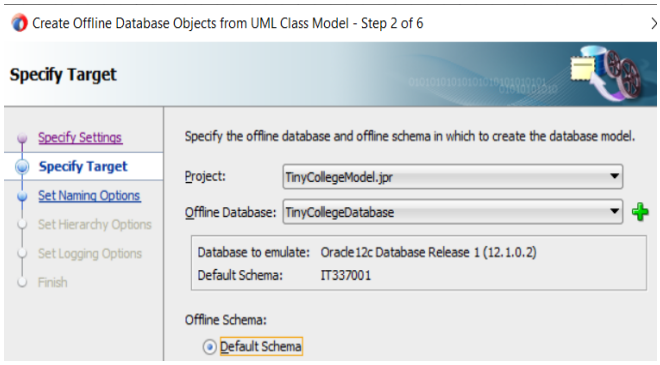
d. In the Step 2 (Create Offline Database Objects from UML Class Model dialog), click the **Create Offline Database** button.



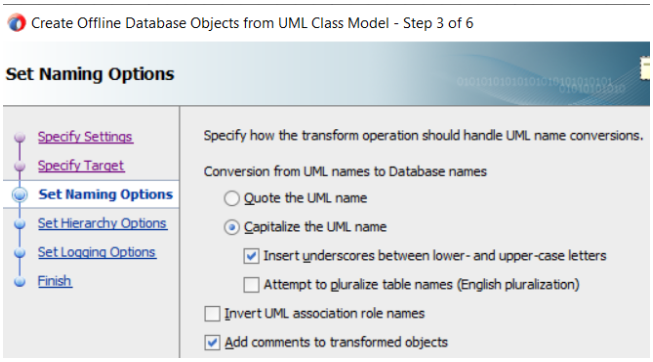
e. In the **Create Offline Database** dialog, enter the **Name** (e.g., TinyCollegeDatabase) and **Default Schema** (e.g., IT337001). Go back to the previous dialog (Step 2 of 6), select **Default Schema** option under Offline Schema.



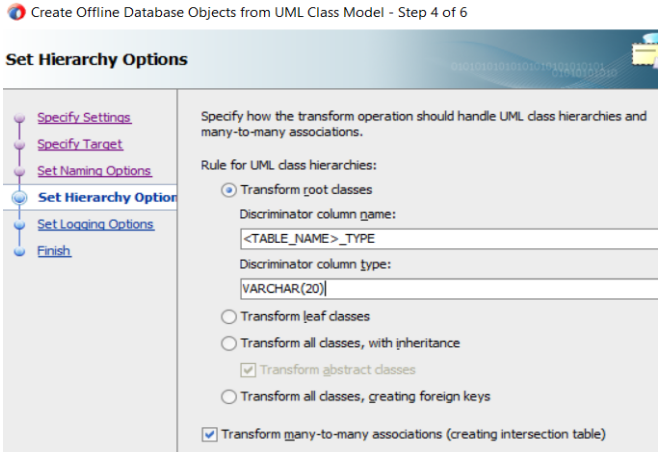
f. Step 2 (Specify Target) should now look like this.



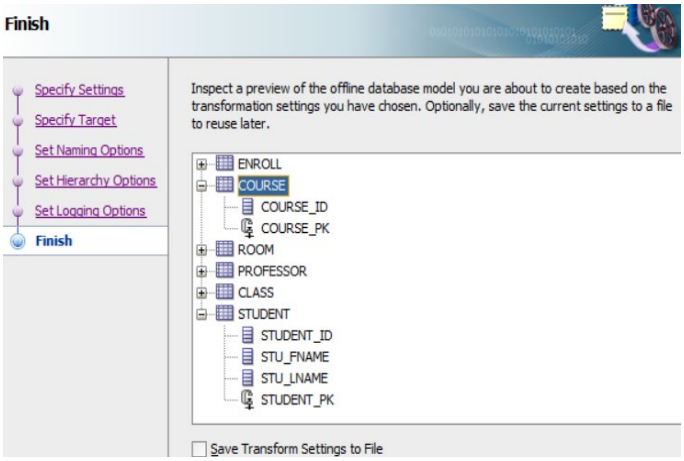
g. Under **Set Naming Options**, check the shown check boxes (default options).



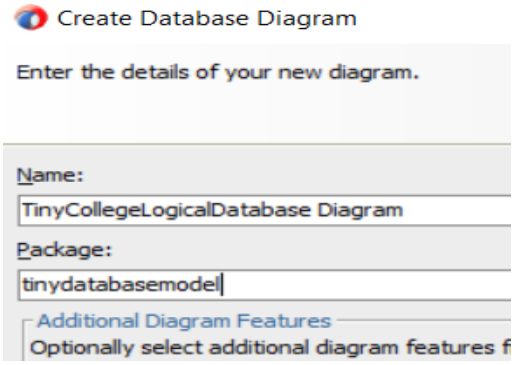
h. Select the **Transform root classes**. Consider changing Column Type from the default of (CHAR) data type / (30) data size. Notice the checked box about transformation of many to many associations (at the bottom).



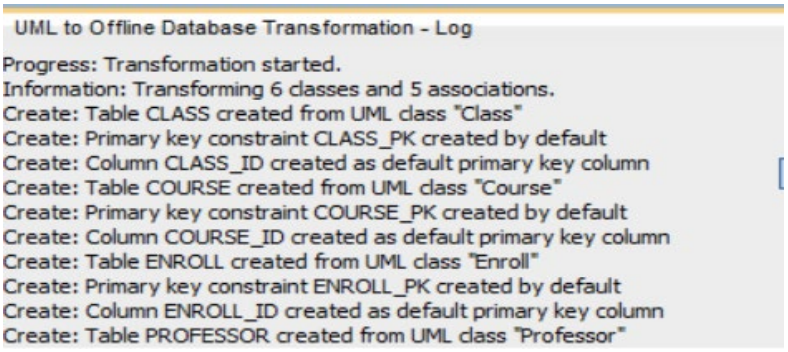
i. The **Finish** shows the preview of database model that is about to be transformed. You can expand the nodes for further details (e.g., primary keys, etc.). Click **Finish**.



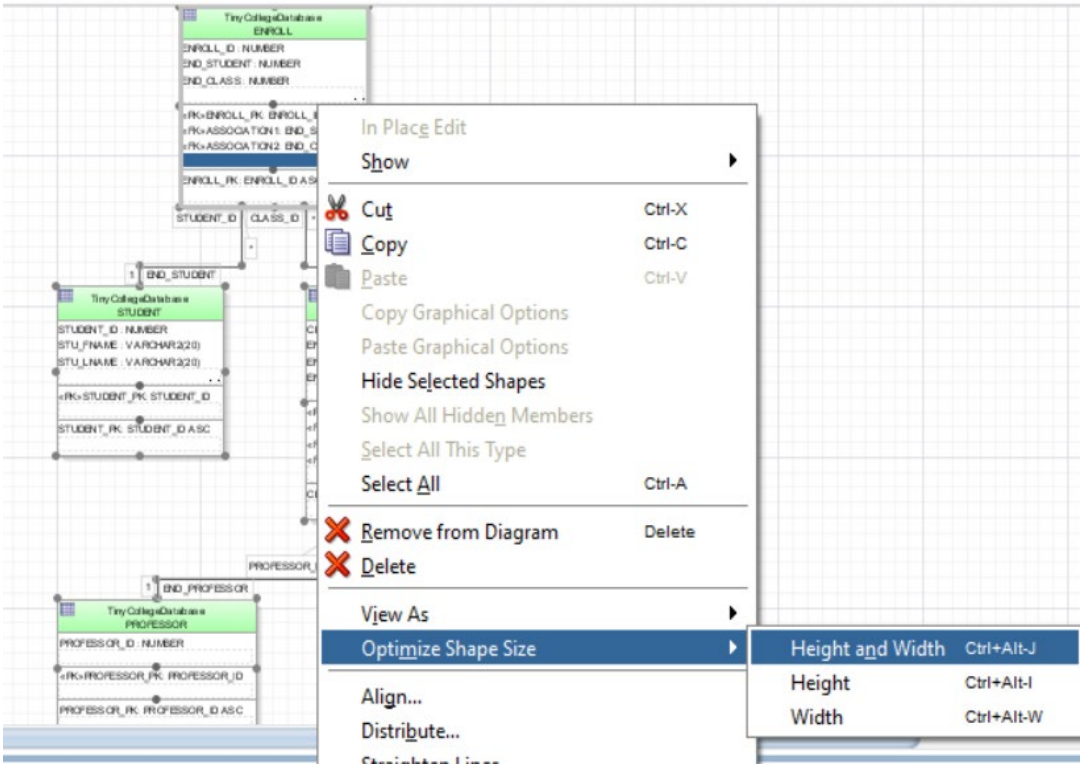
j. Now enter the **Name** for your new diagram (e.g., TinyCollegeLogicalDatabaseDiagram) and leave the **Package** name default.



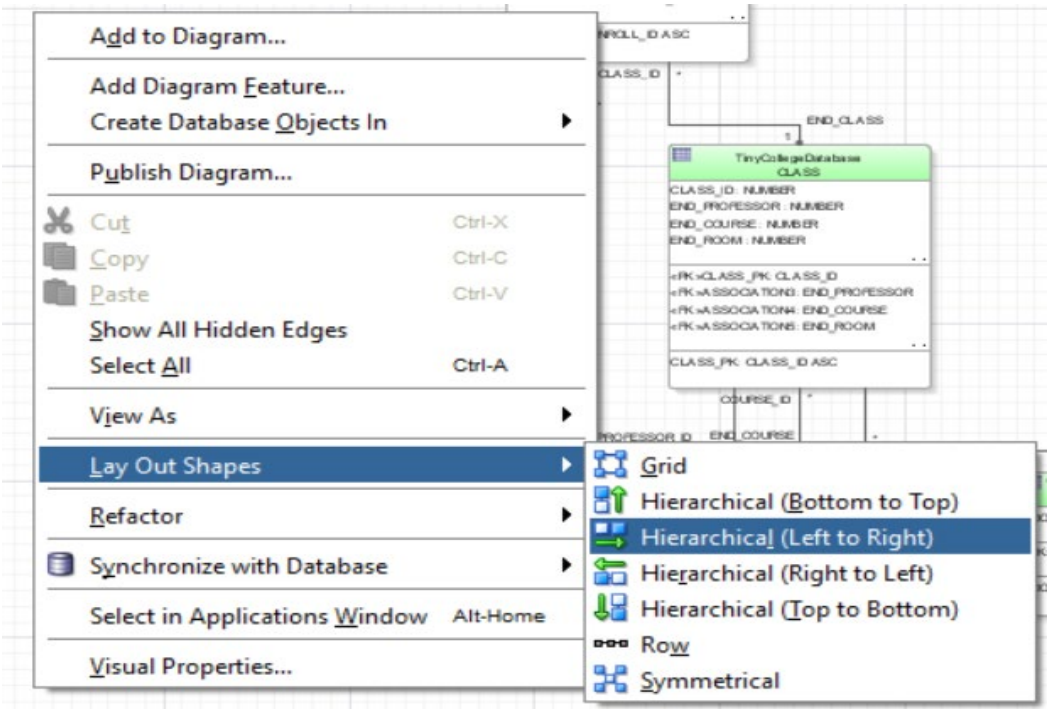
k. The transformation log will have the details about the transformation.



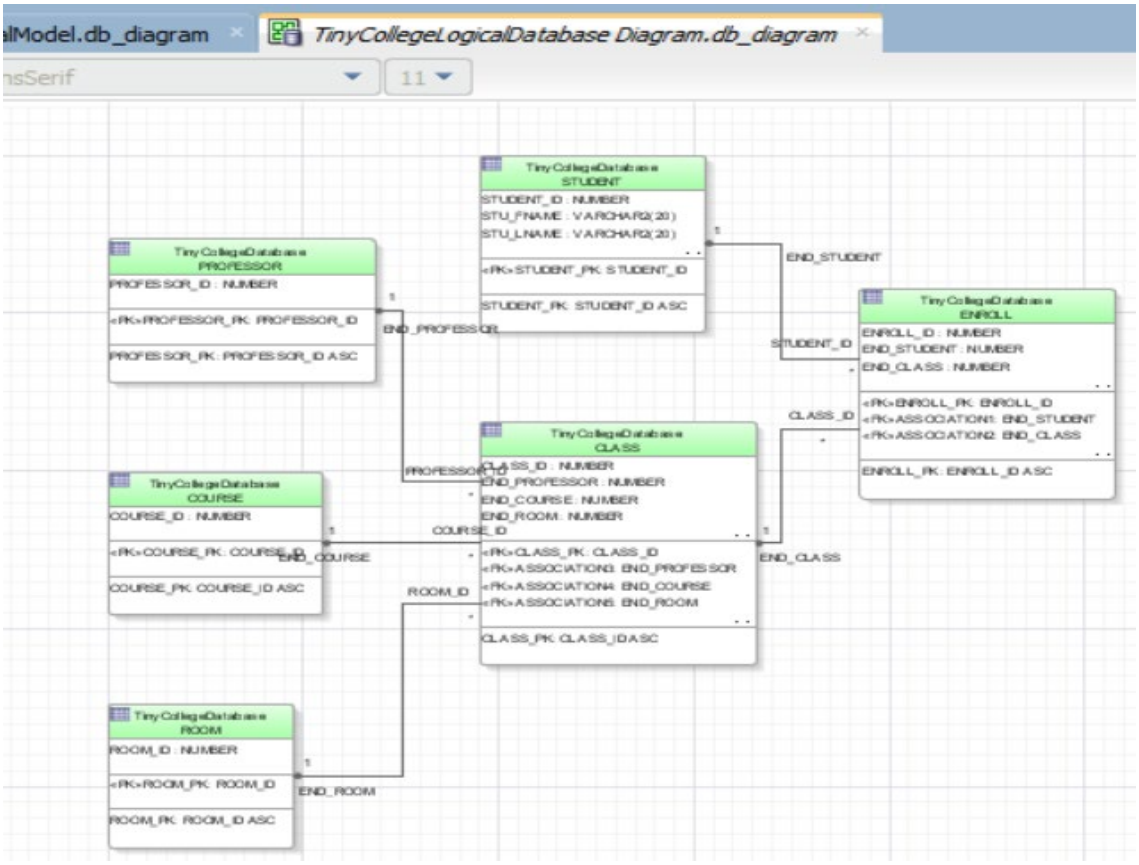
l. From the main menu, select **Edit --> Select All** then right click in any table and select **Optimize Shape Size --> Height and Width** to improve the default diagram layout.



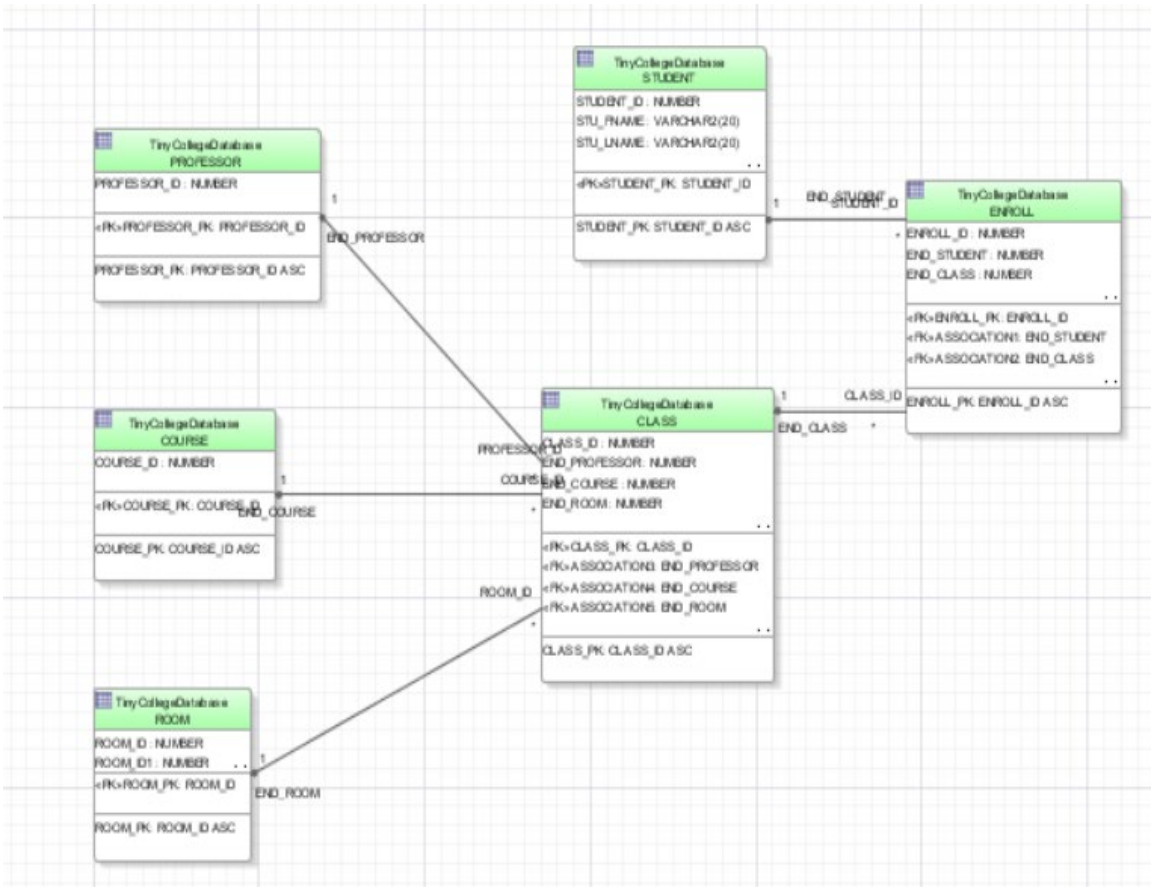
m. For a new presentation look Right click on diagram surface and choose **Lay Out Shapes --> Hierarchical (e.g. Left to Right)** option.



n. Your diagram should present itself somewhat like this:



o. To straighten all/most association lines in a single attempt, Right click on an association and from the context select **Select All This Type,** right click again, and select **Straighten Lines**.



Some Observations about the Transformation process:

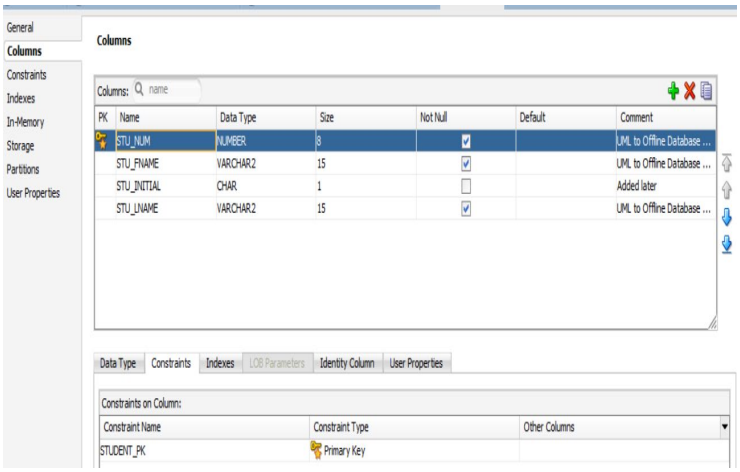
- The transformation process automatically generated primary key attributes for all the tables using the format TABLENAME_ID (e.g., COURSE_ID). As these attributes are the primary keys (e.g. COURSE_PK:COURSE_ID) a stereotype <<PK>> is used to flag this (refer to constraints section). These PKs which are IDs can however also be changed (e.g. attribute name Student_ID can be changed to name Stu_Num or the key itself can also be changed as in the Enroll table - instead of using auto-generated Enroll_ID as the PK, a composite* PK is used (*composite key is a combination of attributes) and in this case is a combination of FKs from Student and Class tables – this as shown in the modified logical diagram).

- The foreign key attributes are also being added (e.g., END_COURSE) using a similar format (e.g., END_TABLENAME). In case a table has multiple FKs, a suffix of the parent table is added after the word END (e.g., END_STUDENT, END_CLASS in ENROLL). The FK constraint names are <<FK>> ASSOCIATION1: END_STUDENT and <<FK>> ASSOCIATION2: END_CLASS. The attribute and constraint names should be altered to suit the design domain requirements and naming conventions.

- Select the modelling element (e.g., table, association, etc.) and either double-click or R-click to access the element's properties.

- The default datatype for the PK attribute is Number. The String type from the class diagram here maps to Oracle Varchar2(20). The datatypes and sizes should be altered as per the design requirements.

- Modify the data model diagram by altering default attribute names (e.g., STUDENT_ID changed to STU_NUM), adding missing attributes (e.g., STU_INITIAL) and their appropriate data types, sizes, etc. (e.g., Varchar2(15)). The order of the attributes in a table can also be changed by moving these up or down.



- Whether a table column (attribute) is required or not is flagged using Not NULL property (e.g., Stu_Initial is an optional attribute).

- Add any the missing associations (e.g., association between Professor and Student would store information about which Professor is advising a Student if that were the case requirement). Use foreign key from Database Objects to add the FK association if necessary. Make sure that in case of the attributes that are foreign keys, their data types and sizes are consistent with the values in the parent tables (the case tool does not seem to object if inconsistent).

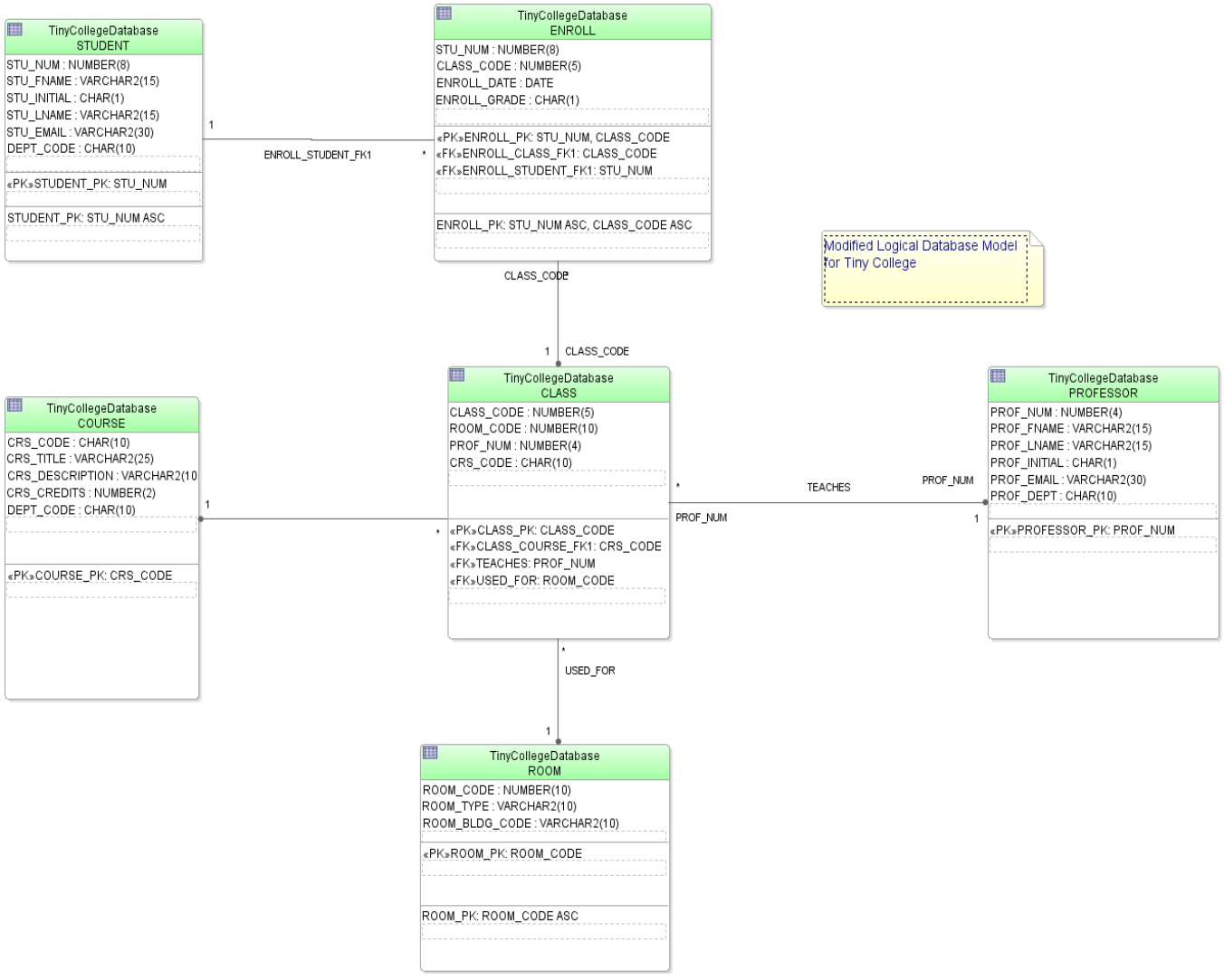
- Add any missing tables.

- UML notes can be used to clarify/highlight/label elements.

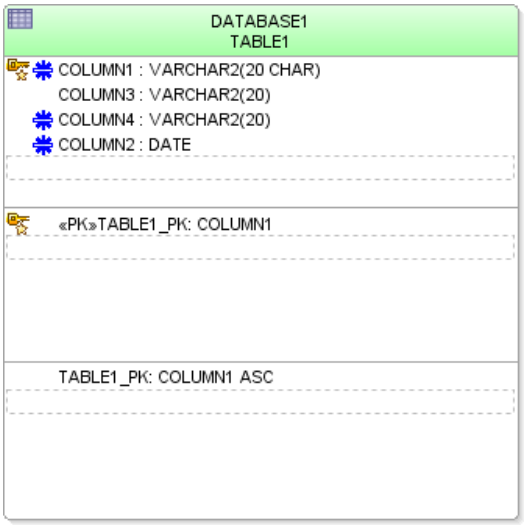
Note: Always check your models for correctness, consistency and make appropriate changes as required.

- Modified data model diagram should look like the figure shown. You can change the Visual Properties of the diagram elements (table, association, etc):
    - table constraints displayed for all the tables, but indexes are not shown for all.
    - associations have varying visual properties e.g., association name shown as default – ENROLL_STUDENT_FK and changed to TEACHES, USED_FOR and not shown as well; association showing columns – (CLASS_CODE, PROF_NUM, etc.) for some and not showing for the others, etc.

Modified Logical Data Model Diagram for Tiny College



Note: Here the properties of as table are changed to show required/not required attributes, keys, etc using *icons* (Check Show Icons under Visual properties after selecting a table and invoking context related menu).



Part b

b. **SQL based Tutorial 2 – (Relational Set Operators)**

i. Download the **Chapter3_StoreCO-ORA** script file from week2 of the course Stream site.

ii. Run this script file under SQL developer to create and populate the tables (if your account already has any tables with the same names then either drop or rename those tables for this script to run successfully).

Notes (read these before you attempt iii. below):

- No constraints exist for the tables in the given script file. For some of the questions below you must add asked constraints (e.g., add the PKs). Make the required changes to the code in notepad. Before you run the revised code for a table, you will need to the drop table, as the table now already exists in your Oracle a/c. You can either use the drop table statement (e.g., drop table employee cascade constraints;) or use GUI (i.e., select a table in the Application Navigator and r-click and select Table →Drop from the context related menu) to drop a table.

- Code for most of the constraints can be found from the lecture slides 21-24 (lecture 2).

- For (iii c.) below read the following information about NUMBER data types:
  For numeric columns, you can specify column as:
  column_name NUMBER

  Optionally, you can specify a **precision** (total number of digits) and **scale** (number of digits to the right of the decimal point) as:
  column_name NUMBER (precision, scale)

- Give meaningful names to the constraints when appropriate.

iii. Now attempt the following questions based on this database:

a. Identify primary keys for each of the given three tables.
b. Change the given SQL scripts to mark the identified columns as primary keys.
c. Format the column STORE_YTD_SALES to allow it to accept the values with **two** decimal places e.g., 2342200.55.
d. Add a constraint indicating that the column REGION_DESCRIPT cannot be null.
e. Add a constraint that STORE_YTD_SALES value cannot be less than 1000.
f. Write a simple SQL statement using the UNION Operator between STORE and EMPLOYEE tables.
g. Write a SQL statement to display the names of employees who are store managers. The complete employee's name (i.e., employee title, first name,

last name) and the store name should both be listed. Which join will you use?

h. Write a SQL statement to display the names of the employees (i.e., employee title, first name, last name) who do not manage stores.
i. How many rows will be returned when a Cartesian product is taken between STORE and REGION? Also, check the result set with the COUNT operator.
j. Show that INTERSECT and EQUIJOIN operators will return the same result set.

Part c

c. Analyse some relational operators using the script provided.

i. Create and populate three tables named t1, t2 and t3 using the following script:

```
prompt "my joins"
create table table1 (
  col1 number);
create table table2 (
  col2 number);
create table table3 (
  col3 number);

insert into table1 values (1);
insert into table1 values (2);
insert into table1 values (3);

insert into table2 values (2);
insert into table2 values (3);
insert into table2 values (4);

insert into table3 values (2);
insert into table3 values (3);
insert into table3 values (5);

commit;
```

ii. Analyse the relational operators used in each of the following SQL statements. Execute these statements and look at the returned query result sets to understand how these operators work.

a. select count(*) from table1, table2, table3;

b. select count(*) from table1, table2;

c.
```
select col1 from table1
union
select col2 from table2;
```

d.
```
select col1 from table1
union all
select col2 from table2;
```

e.
```
select table1.col1, table2.col2
from table1 join table2
on (col1 = col2);
```

f.
```
select table1.col1, table2.col2
from table1, table2
where col1 = col2;
```

g.
```
select col1 from table1
union all
select col2 from table2
minus
select col3 from table3;
```

h.
```
select col1 from table1
union all
(select col2 from table2
minus
select col3 from table3);
```